

Graphs

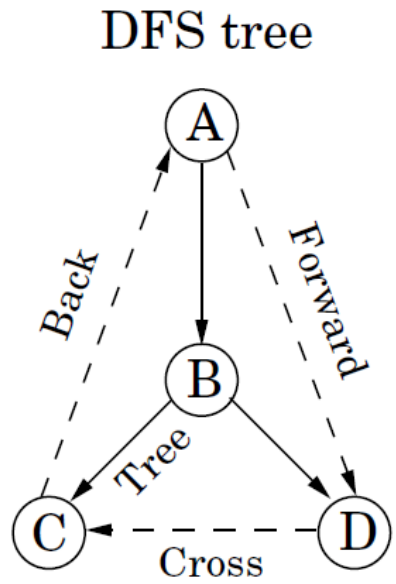
Types of edges:

Tree edges are actually part of the DFS forest.

Forward edges lead from a node to a *nonchild* descendant in the DFS tree.

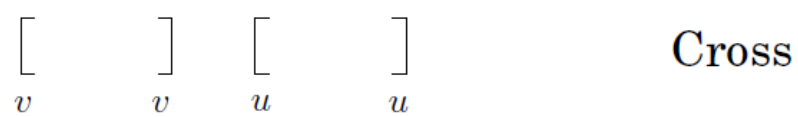
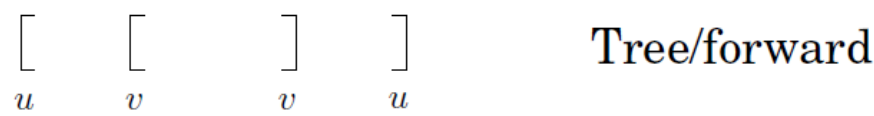
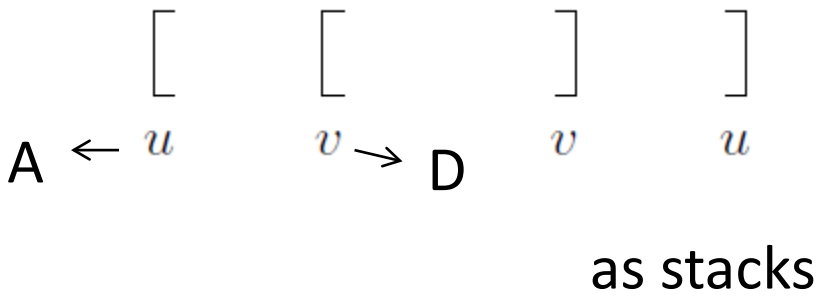
Back edges lead to an ancestor in the DFS tree.

Cross edges lead to neither descendant nor ancestor; they therefore lead to a node that has already been completely explored (that is, already postvisited).



$$\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u)$$

Ancestor and descendant relationships



$$(u, v)$$

Directed Acyclic Graphs

cycle $\rightarrow$ directed graph is a circular path

$$v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_k \rightarrow v_0$$

A graph without cycles is *acyclic*



Digraph

Trees?, not really, more generic!

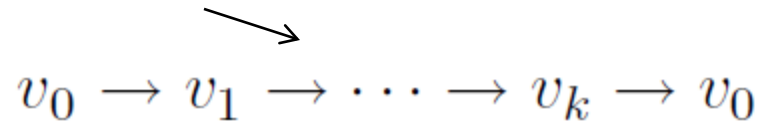
How do you know that a digraph G is cyclic or not?: DFS! linear time!

Property: A directed graph has a cycle if and only if its depth-first search reveals a back edge.

Runtime: $O(|V| + |E|)$

1. If you have a back-edge (u, v) , then you have a cycle.
 - As there will be a path of tree edges from v to u .

2. If you have a cycle, then do you have a back-edge? - yes



Let the first visited (by DFS) vertex in the loop be a $v_i, i \in \{0, \dots, k\}$. Then, v_{i-1} will be visited as there is at least a path from v_i to v_{i-1} . Since, the edge (v_{i-1}, v_i) will be considered while exploring v_{i-1} and that v_i is already visited in the path (of tree edges) to v_{i-1} , (v_{i-1}, v_i) will by definition be marked as a back edge [v_i is an ancestor of v_{i-1} for the DFS at hand].

Note: $v_{i-1} = v_k$, if $v_i = v_0$.

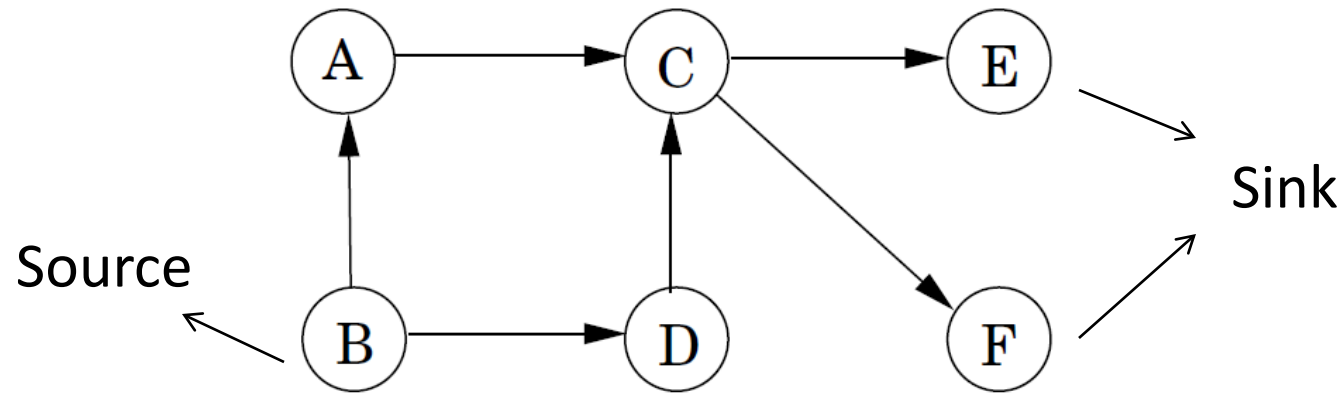
With both (1) and (2), the property is proved.

Directed acyclic graphs (DAGs)

$\Downarrow$
causalities, hierarchies, and temporal dependencies

DAGs $\longrightarrow$ *linearize (or topologically sort)*

We can order the vertices one after another (in more than 1 way)



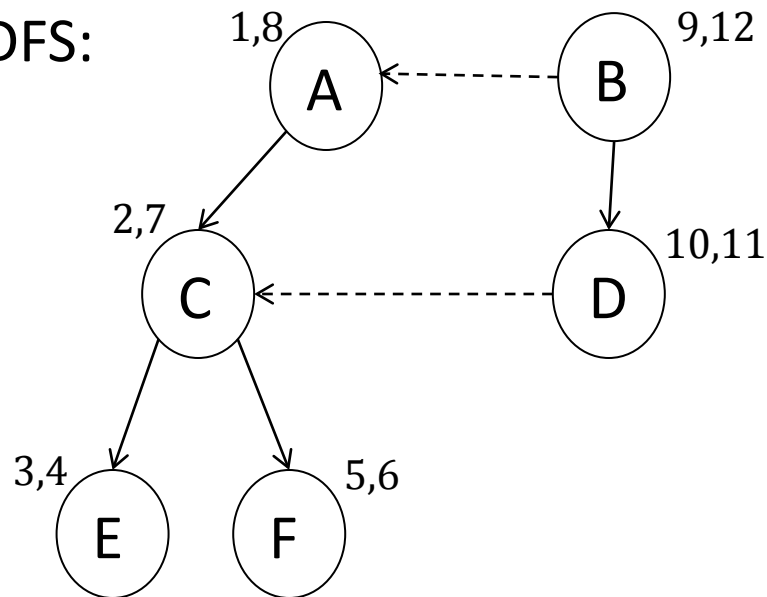
B,A,D,C,E,F or B,A,D,C,F,E or B,D,A,C,F,E or B,D,A,C,E,F

Topological Sort: Linear time!

Depth-first search- Perform tasks (vertex) in decreasing order of their post numbers.

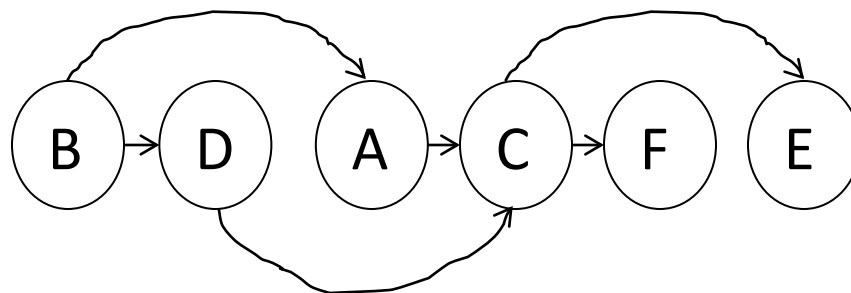
In DAG, Lemma: $(u, v) \in E \rightarrow post(u) > post(v)$ 2 cases

DFS:



Using post numbers:

B,D,A,C,F,E



Property: In a DAG, every edge leads to a vertex with a lower post number.

Acyclicity = linearizability = absence of back edges in a DFS.

Highest post must be a source in DAG & smallest post must be a sink

Property: Every DAG has at least one source and at least one sink.

Connectivity for Digraphs:

In undirected graphs, two vertices u & v are connected if there is a path between them.

So, a graph that is not connected can be decomposed in a natural and obvious manner into several connected components.

But:

Two nodes u and v of a directed graph are *connected* if there is a path from u to v and a path from v to u .

Different paths (edge set)!

Strongly connected!

Weakly connected?!

Obs: if both $(u, v) \in E$ & $(v, u) \in E$, two vertices u & v are strongly connected (converse not true)

We now have equivalence relation like before!

This relation partitions V into disjoint sets

strongly connected components

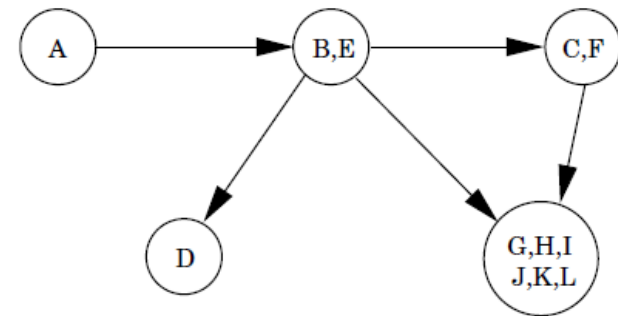
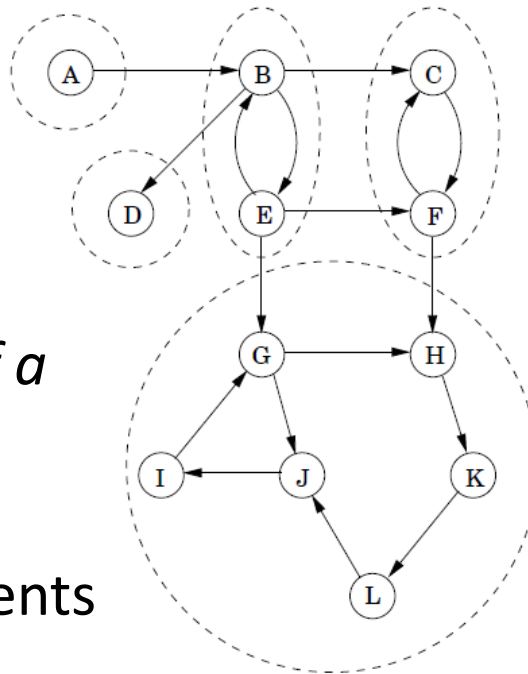
Property Every directed graph is a DAG of its strongly connected components.*

All strongly connected components in the digraph

→ Meta-vertices/nodes

Draw an edge from one meta-node to another if there is an edge (in the same direction) between their respective components.

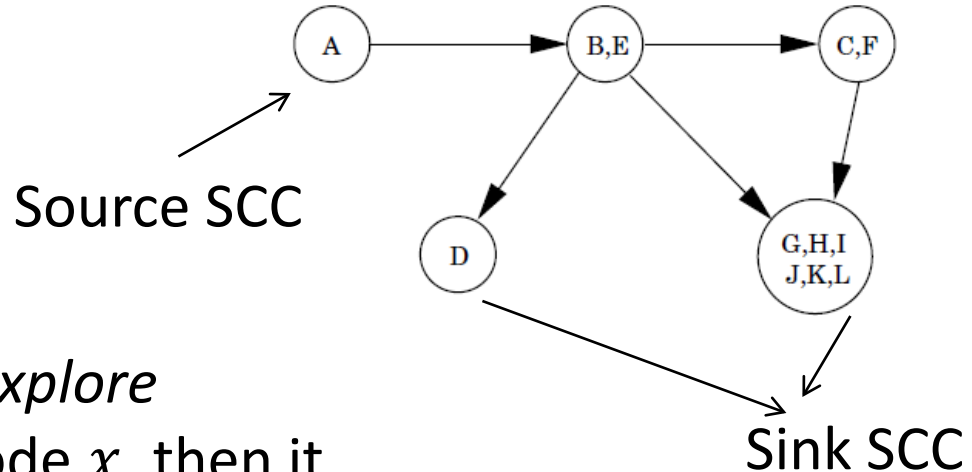
↓
Meta-graph
↓
DAG



The connectivity structure of a directed graph is two-tiered:
DAG at top (coarse) level,
strongly connected components
at lower (fine) level.

ALGO: to get strongly connected component (SCC)

What happens
when we use
explore within
DFS?



Property (known): If the *explore* subroutine is started at node x , then it will terminate precisely when all nodes reachable from x have been visited.

How about recursively finding sink SCCs and applying explore?

We need to a way to find a sink (or source) SCC!

Given digraph G^R (reversed G), find any vertex
in source SCC.

↖ ↗
All edge reversed!

Property The node that receives the highest post number in a depth-first search must lie in a source strongly connected component.

After DFS, the vertex with largest post number is going to be part of source SCC!

Property If C and C' are strongly connected components, and there is an edge from a node in C to a node in C' , then the highest post number in C is bigger than the highest post number in C' .

If a path from C to C' exists, there won't be a path from C' to C .

- If the DFS visits a vertex in C before any in C' , then clearly all of C and C' will be traversed before the procedure gets stuck. So C will contain the highest post number.
- If the DFS visits a vertex in C' before any in C , then the procedure gets stuck after traversing all of C' . So C will contain the highest post number.

→ The strongly connected components can be linearized by arranging them in decreasing order of their highest post numbers. DAG (generalized)!

Again, the procedure was to *recursively find sink SCCs and applying explore.*



Just use the post numbers of vertices not visited yet after applying explore, in decreasing order!

Will conclude when no more nodes are left!

The ALGO:

↗ *Linear!*

1. Get G^R from G .
2. Run DFS on G^R to get the post numbers

*Like
applying
DFS twice!
Linear!*

3. $\forall v \in V$ in reverse post # order of G^R :

if not visited $[v]$: $\longrightarrow$ NULL initialization

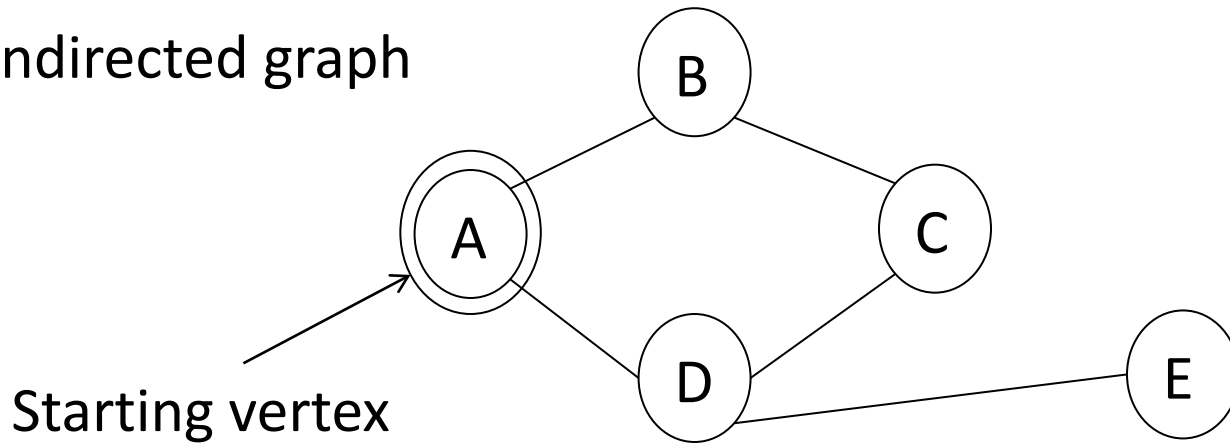
scc++ $\longrightarrow$ 0 initialization

explore (G, v) $\rightarrow$ SCC count storage with

NULL initialization

Distances, Shortest Paths

Undirected graph



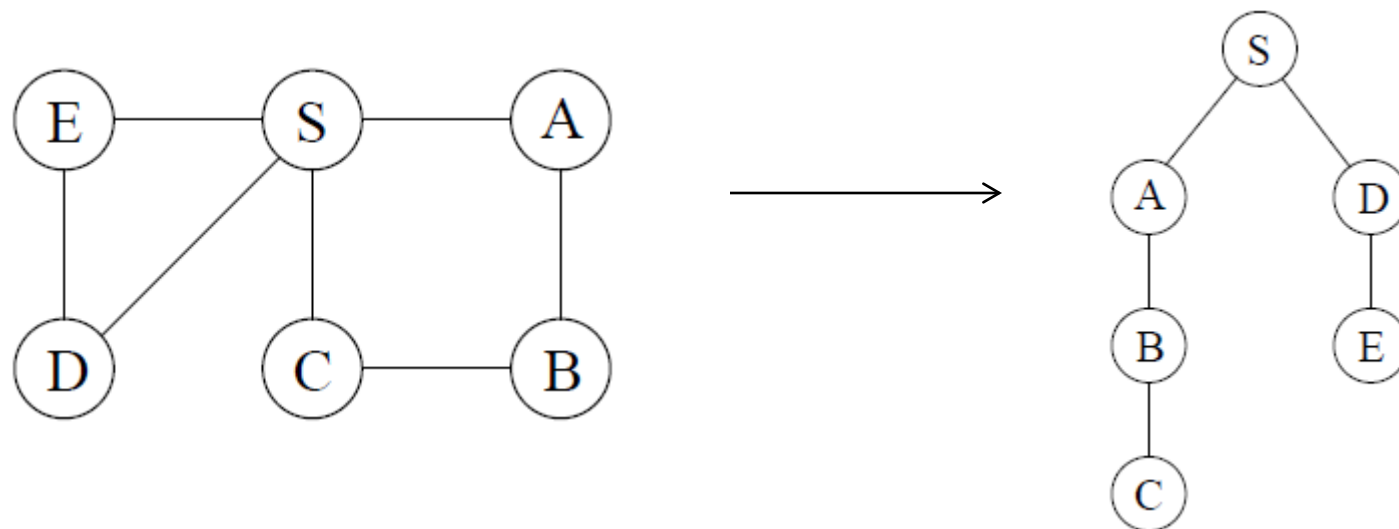
Assume that all edge cost (distance) is the same, say, 1

Distances from A (intuitively):

B: 1	C: 2
D: 1	E: 2
A: 0	

The *distance* between two nodes is the length of the shortest path between them.

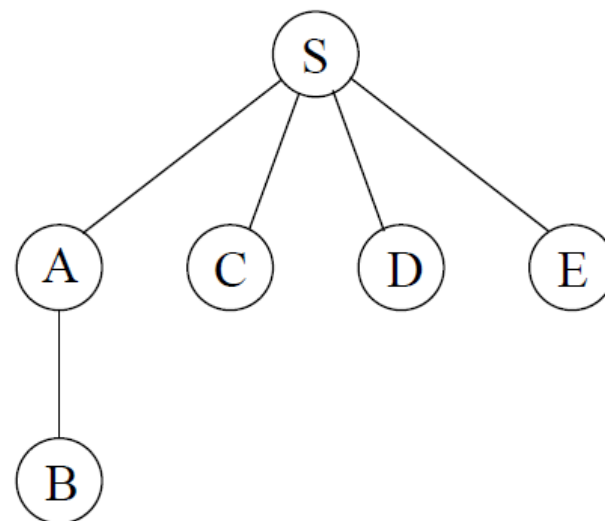
DFS checks connectivity, but may not quantify distance!



Because we are going down the depth... and not considering all the adjacent vertices one after the other (breadth).*

Breadth First Search

adjacent to the layer at distance d
distance $0, 1, 2, \dots, d \xrightarrow{\uparrow} d+1$



Breadth-first search.

procedure `bfs( $G, s$ )`

Input: Graph $G = (V, E)$, directed or undirected; vertex $s \in V$

Output: For all vertices u reachable from s , `dist( $u$ )` is set to the distance from s to u .

for all $u \in V$: } $|V|$
`dist( $u$ ) =  $\infty$`

`dist( $s$ ) = 0`

`$Q = [s]$` (queue containing just s)

while `$Q$  is not empty:` } Worst case $|V|$
 `$u = \text{eject}(Q)$` *visit

for all edges $(u, v) \in E$: } Worst case $|E|$
 if `$\text{dist}(v) = \infty$` :
`inject( $Q, v$ )`
 `$\text{dist}(v) = \text{dist}(u) + 1$`

Order of visitation	Queue contents after processing node
	$[S]$
S	$[A \ C \ D \ E]$ → distances
A	$[C \ D \ E \ B]$ → recorded
C	$[D \ E \ B]$
D	$[E \ B]$
E	$[B]$
B	$[\]$

Time complexity:

$2|V|$

$2|E| \rightarrow$ undirected

Linear $\rightarrow O(|V| + |E|)$

Inject, eject, add, comparison, assignment all base operations

All nodes & edges may not be visited!

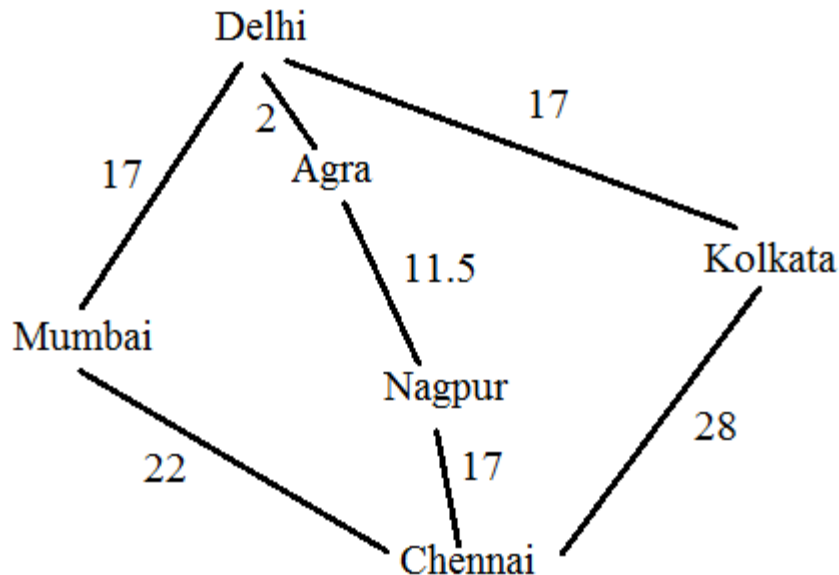
Lengths (costs /weights) in edges



Finding shortest path!

$$e = (u, v) \longrightarrow l(u, v) \text{ or } l_{uv}$$

Assume non-negative



Node Queue : Distances

$[D]: [0]$

$[M \ A \ K]: [17 \ 2 \ 17]$

$[A \ K \ C]: [2 \ 17 \ 39]$

$[K \ C \ N]: [17 \ 39 \ 13.5]$

$[C \ N]: [39 \ 13.5]$

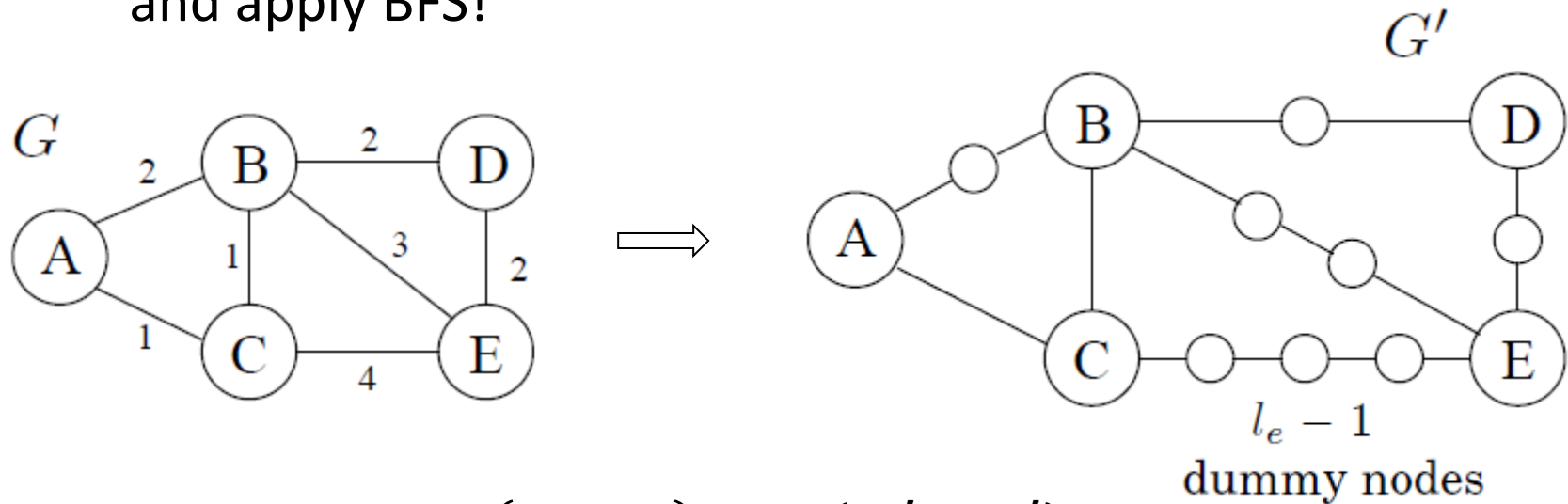
$[N]: [13.5]$

$[]: []$

$C \text{ from } D = 39 \longrightarrow \text{Not the shortest!}$

Number of hops mostly is not what matters in real life!

How about breaking down an edge with **positive integer length** into **multiple edges of unit length**, and apply BFS!



$$O(m + n) \rightarrow O(m' + n')$$

Something we can do to quickly traverse over dummy nodes?

Dijkstra's algorithm

Dijkstra's Algorithm:

Dijkstra's shortest-path algorithm.

procedure `dijkstra`(G, l, s)

Input: Graph $G = (V, E)$, directed or undirected;
positive edge lengths $\{l_e : e \in E\}$; vertex $s \in V$

Output: For all vertices u reachable from s , `dist`(u) is set
to the distance from s to u .

*Obtaining the distance, is same as obtaining
the shortest path if the traversals leading to
the shortest distances are stored!*

State: $\longrightarrow$ Changes (updated) during the
course of the algorithm

1. Distance estimates: $\text{dist}(u), \forall u \in V$

2. Known vertices: $R \subseteq V$ where $\text{TRUE } \text{dist}(u), \forall u \in R$

$\nwarrow$
The distance estimate is the true
distance.